

Lecture 8: ECE C147A/C247A, Neural Networks & Deep Learning

Initialization, regularization and optimization
October 21st, 2026

These slides are based on material from Professor Jonathan Kao, who also developed it based on other freely available sources including from Stanford and other resources, whom we acknowledge.

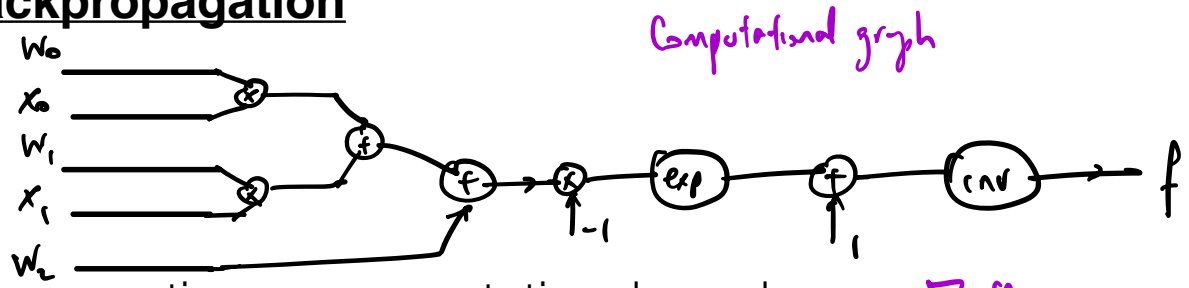
Announcements

- HW #3 is due Friday October 23, 11.59pm uploaded to Gradescope. **Note that even 1 minute late is considered a day late! Your responsibility to confirm upload of HWs**
- HW#1, 2 solutions posted.
- Midterm 4th November 2026 (MSOL Midterm November 7th, Saturday); allowed one sheet (letter size), no devices or phones allowed in exam. We will have 2 rooms for the exams and will announce who will go where.
- Practice midterms will be posted soon, and solutions posted a week later.
- **Please read the rules handout carefully including for late HWs etc.**
- To submit your Jupyter Notebook, print the notebook to a pdf with your solutions and plots filled in. The graders will grade your work from these notebooks. In the future, you will also have to print out code from any .py files that were modified.
- Discussion sessions held on Fridays 11am, will be recorded and live zoom link also is available. → Oct 30th discussion midterm review
- For midterm **CAE approved accommodation students have to contact CAE office to proctor exam** as we cannot do this ourselves. Please do this pro-actively.

Review of previous lecture

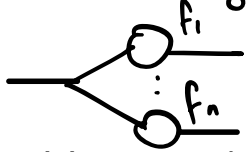
- **Computational graphs and backpropagation**

$$f(W, X) = \frac{1}{1 + \exp\left(-\left[W_0 X_0 + W_1 X_1 + W_2\right]\right)}$$



  - Forward and backward propagation on computational graphs.
  - Gating viewpoint of operators.
  - Extending back propagation through neural networks: vector, matrix gradients

- Parallel paths $\rightarrow \sum \frac{\partial f_i}{\partial x} \cdot \frac{\partial L}{\partial f_i}$



$\nabla_x y \rightarrow$ gradient

$$\nabla_A y = \begin{bmatrix} \partial y/\partial a_{11} & \cdots & \partial y/\partial a_{1n} \\ \vdots & & \\ \partial y/\partial a_{m1} & \cdots & \partial y/\partial a_{mn} \end{bmatrix}$$

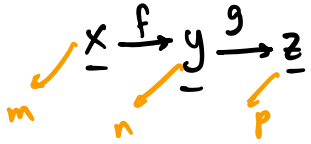
denominator form

$$\nabla_x y = \begin{bmatrix} \partial y_1/\partial x_1 & \cdots & \partial y_n/\partial x_1 \\ \vdots & & \\ \partial y_1/\partial x_m & \cdots & \partial y_n/\partial x_m \end{bmatrix} = J^T$$

$\nabla_x y_k$ — Jacobian

$$\Delta y \approx J \Delta x = (\nabla_x y)^T \Delta x$$

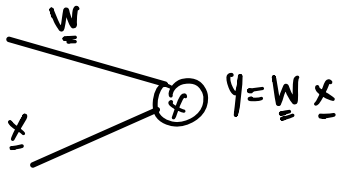
  - Vector chain rule



$$\nabla_x z = (\nabla_x y)(\nabla_y z)$$ (m×p = m×n · n×p) → ORDER MATTERS!

  - Tensor derivatives and application to backpropagation.

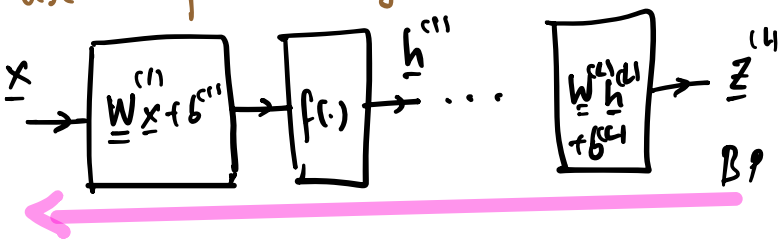
Tensor derivative



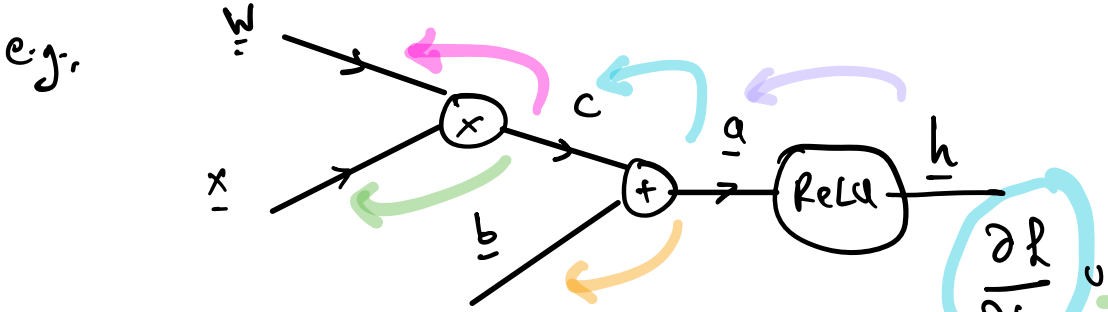
$$\nabla_x y = W^T \Rightarrow \frac{\partial L}{\partial x} = W^T \frac{\partial L}{\partial y}$$

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial y} \cdot x^T$$ → not obvious, will step through

Use these for BP through NN layers



Review of previous lecture (Contd.)



$\text{relu}(a) = \max(a, 0)$

$\frac{\partial (\cdot)}{\partial a} = \mathbb{1}_{\{a>0\}}$

$h = \text{ReLU}(a) = \begin{bmatrix} \text{relu}(a_1) \\ \vdots \\ \text{relu}(a_n) \end{bmatrix}$

$\frac{\partial h}{\partial a_j} = \mathbb{1}_{\{a_j \geq 0\}} e_j$ ← unit vector

for componentwise $\frac{\partial h}{\partial a} = \mathbb{1}_{\{a \geq 0\}} \frac{\partial \ell}{\partial h}$

$$\frac{\partial \ell}{\partial c} = \frac{\partial \ell}{\partial b} = \frac{\partial \ell}{\partial a}$$

$$\frac{\partial \ell}{\partial W} = \frac{\partial \ell}{\partial c} x^T$$ → non obvious fact stated before

$$\frac{\partial \ell}{\partial x} = W^T \frac{\partial \ell}{\partial c}$$

"local gradient"

$\frac{\partial u}{\partial V}$: $u \in \mathbb{R}^p$, $V \in \mathbb{R}^{m \times n}$, $\frac{\partial u}{\partial V} \in \mathbb{R}^{m \times n \times p}$ TENSOR

$\frac{\partial u_k}{\partial V_{ij}}$

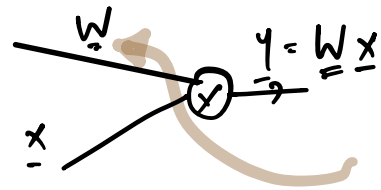
Multivariate chain rule and tensor derivative example

Consider the squared loss function:

$$\mathcal{L} = \frac{1}{2} \sum_{i=1}^{N} \left\| \mathbf{y}^{(i)} - \mathbf{W}\mathbf{x}^{(i)} \right\|^2$$

Here, $\mathbf{y}^{(i)} \in \mathbb{R}^m$ and $\mathbf{x}^{(i)} \in \mathbb{R}^n$ so that $\mathbf{W} \in \mathbb{R}^{m \times n}$. We wish to find $\mathbf{W}$ that minimizes the mean-square error in linearly predicting $\mathbf{y}^{(i)}$ from $\mathbf{x}^{(i)}$.

GOAL



$\frac{\partial \ell}{\partial y}$ Naturally $\frac{\partial \ell}{\partial W}$ natural $\frac{\partial y}{\partial W} \cdot \frac{\partial \ell}{\partial y}$ → upstream

local

$y = W x$ $\quad W = \begin{bmatrix} W_{11} & \cdots & W_{1n} \\ \vdots & & \vdots \\ W_{m1} & \cdots & W_{mn} \end{bmatrix}$ $\quad x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$

$$y = \begin{bmatrix} W_{11} & \cdots & W_{1n} \\ \vdots & & \\ W_{m1} & \cdots & W_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \Rightarrow y_k = \sum_{j=1}^{n} W_{kj} x_j$$

Next Calculate $\frac{\partial y_k}{W_{ip}}$



Multivariate chain rule example

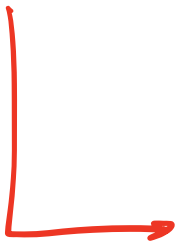
NOT TESTED

We consider one example, $\varepsilon^{(i)} = \frac{1}{2}\left\|\mathbf{y}^{(i)} - \mathbf{W}\mathbf{x}^{(i)}\right\|^2$.

We wish to calculate $\nabla_{\mathbf{W}}\varepsilon^{(i)}$.

To do so, we define $\mathbf{z}^{(i)} = \mathbf{y}^{(i)} - \mathbf{W}\mathbf{x}^{(i)}$. Then, $\varepsilon^{(i)} = \frac{1}{2}(\mathbf{z}^{(i)})^T\mathbf{z}^{(i)}$. For the rest of the example, we're going to drop the superscripts (i) and assume we're working with the ith example (to help notation). Then, we need to calculate is:

$$\nabla_{\mathbf{W}}\varepsilon = \nabla_{\mathbf{W}}\mathbf{z}\nabla_{\mathbf{z}}\varepsilon$$



read on your own



Multivariate chain rule example

NOT TESTED

Now, we need to calculate $\nabla_{\mathbf{W}}\mathbf{z}$. This is a three dimensional tensor with dimensionality $m \times n \times m$.

This makes sense dimensionally, because when we multiply a $(m \times n \times m)$ tensor by a $(m \times 1)$ vector, we get out a $(m \times n \times 1)$ tensor, which is equivalently an $(m \times n)$ matrix. Because $\nabla_{\mathbf{W}}\varepsilon$ is an $(m \times n)$ matrix, this all works out.

↳ read on your own

Multivariate chain rule example

NOT TESTED

$$\underline{y} = \begin{bmatrix} w_{11} & \cdots & w_{1n} \\ \vdots & & \\ w_{m1} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \Rightarrow y_k = \sum_{j=1}^{n} w_{kj} x_j$$

$$\delta_\ell = \begin{cases} 1 & \ell = 0 \\ 0 & \text{else} \end{cases}$$

$$\hookrightarrow x_p \, \delta_{i-k} \, \delta_{j-p}$$

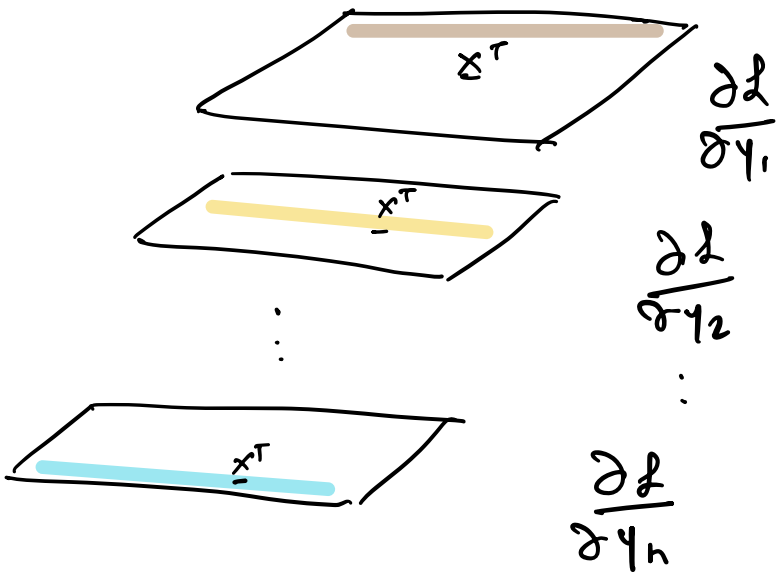
$$\frac{\partial y_k}{\partial \underline{W}_{ip}} = \sum_{j=1}^{n} \frac{\partial}{\partial w_{ip}} \left[w_{kj} x_j \right]$$

$$\frac{\partial}{\partial w_{ip}} \left[w_{kj} x_j \right] = \begin{cases} 0 & i \neq k \\ 0 & i = k, \; j \neq p \\ x_p & i = k, \; j = p \end{cases}$$

$$\frac{\partial y_k}{\partial \underline{W}} = \begin{bmatrix} 0 \\ \underline{x}^T \\ 0 \end{bmatrix} \leftarrow k^{th} \text{ row}$$

$$\frac{\partial \ell}{\partial \underline{W}} = \sum_{k=1}^{n} \frac{\partial y_k}{\partial \underline{W}} \cdot \frac{\partial \ell}{\partial y_k}$$

$$= \begin{bmatrix} \frac{\partial \ell}{\partial y_1} \underline{x}^T \\ \vdots \\ \frac{\partial \ell}{\partial y_n} \underline{x}^T \end{bmatrix} = \left(\frac{\partial \ell}{\partial \underline{y}} \right) \underline{x}^T$$



Multivariate chain rule and tensor derivative example (cont.)

$\nabla_{\mathbf{W}}\mathbf{z}$ is an $(m \times n \times m)$ tensor.

- Letting z_k denote the kth element of $\mathbf{z}$, we see that each $\frac{\partial z_k}{\partial \mathbf{W}}$ is an $m \times n$ matrix, and that there are m of these for each element z_k from $k = 1, \ldots, m$.
- The *matrix*, $\frac{\partial z_k}{\partial \mathbf{W}}$, can be calculated as follows:

$$\frac{\partial z_k}{\partial \mathbf{W}} = \frac{\partial}{\partial \mathbf{W}} \sum_{j=1}^{n} -w_{kj} x_j$$

Read on your own

and thus, the (k, j)th element of this matrix is:

$$\left(\frac{\partial z_k}{\partial \mathbf{W}}\right)_{k,j} = \frac{\partial z_k}{\partial w_{kj}} = -x_j$$

It is worth noting that

$$\left(\frac{\partial z_k}{\partial \mathbf{W}}\right)_{i,j} = \frac{\partial z_k}{\partial w_{ij}} = 0 \qquad \text{for } k \neq i$$

NOT TESTED

Multivariate chain rule and tensor derivative example (cont.)

Hence, $\frac{\partial z_k}{\partial \mathbf{W}}$ is a matrix where the kth row is $\mathbf{x}^T$ and all other rows are the zeros (we denote the zero vector by $\mathbf{0}$). i.e.,

$$\frac{\partial z_1}{\partial \mathbf{W}} = \begin{bmatrix} -\mathbf{x}^T \\ \mathbf{0}^T \\ \vdots \\ \mathbf{0}^T \end{bmatrix} \qquad \frac{\partial z_2}{\partial \mathbf{W}} = \begin{bmatrix} \mathbf{0}^T \\ -\mathbf{x}^T \\ \vdots \\ \mathbf{0}^T \end{bmatrix} \qquad \text{etc...}$$

read on your own

Now applying the chain rule,

$$\frac{\partial \varepsilon}{\partial \mathbf{W}} = \frac{\partial \mathbf{z}}{\partial \mathbf{W}} \frac{\partial \varepsilon}{\partial \mathbf{z}}$$

is a tensor product between an $(m \times n \times m)$ tensor and an $(m \times 1)$ vector, whose resulting dimensionality is $(m \times n \times 1)$ or equivalently, an $(m \times n)$ matrix.

NOT TESTED

Multivariate chain rule and tensor derivative example (cont.)

We carry out this tensor-vector multiply in the standard way.

$$\begin{aligned}
\frac{\partial \varepsilon}{\partial \mathbf{W}} &= \frac{\partial \mathbf{z}}{\partial \mathbf{W}} \frac{\partial \varepsilon}{\partial \mathbf{z}} \\
&= \sum_{i=1}^{m} \frac{\partial z_i}{\partial \mathbf{W}} \left(\frac{\partial \varepsilon}{\partial \mathbf{z}} \right)_i \\
&= \frac{\partial \varepsilon}{\partial z_1} \begin{bmatrix} -\mathbf{x}^T \\ \mathbf{0}^T \\ \vdots \\ \mathbf{0}^T \end{bmatrix} + \frac{\partial \varepsilon}{\partial z_2} \begin{bmatrix} \mathbf{0}^T \\ -\mathbf{x}^T \\ \vdots \\ \mathbf{0}^T \end{bmatrix} + \cdots \\
&\quad + \frac{\partial \varepsilon}{\partial z_m} \begin{bmatrix} \mathbf{0}^T \\ \mathbf{0}^T \\ \vdots \\ -\mathbf{x}^T \end{bmatrix}
\end{aligned}$$

read on your own

NOT TESTED

Multivariate chain rule and tensor derivative example (cont.)

Continuing the work from the previous page...

$$\begin{aligned}\frac{\partial \varepsilon}{\partial \mathbf{W}} &= -\begin{bmatrix} \frac{\partial \varepsilon}{\partial z_1}\mathbf{x}^T \\ \frac{\partial \varepsilon}{\partial z_2}\mathbf{x}^T \\ \vdots \\ \frac{\partial \varepsilon}{\partial z_m}\mathbf{x}^T \end{bmatrix} \\ &= -\frac{\partial \varepsilon}{\partial \mathbf{z}}\mathbf{x}^T\end{aligned}$$



Multivariate chain rule example

NOT TESTED

Hence, with a final application of the chain rule, we get that

$$\begin{aligned} \nabla_{\mathbf{W}} \varepsilon &= -\mathbf{z}\mathbf{x}^T \\ &= -(\mathbf{y} - \mathbf{W}\mathbf{x})\mathbf{x}^T \end{aligned}$$

Setting this equal to zero, we find that for one example,

$$\mathbf{W} = \mathbf{y}\mathbf{x}^T(\mathbf{x}\mathbf{x}^T)^{-1}$$

Summing across all examples, this produces least-squares.

Multivariate chain rule example

$\underline{y} = \underline{\underline{W}}\,\underline{x}$ $\quad \nabla_{\underline{x}}\,\underline{y} = \underline{\underline{W}}^T$

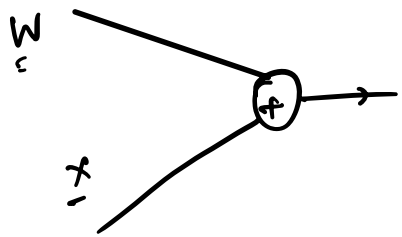
$\nabla_{\underline{\underline{W}}}\, z \rightarrow$ should look like $\underline{x}^T$ something

A few notes on tensor derivatives

$$\underset{m \times n}{\frac{\partial \ell}{\partial \underline{\underline{W}}}} = \underset{m \times 1}{\frac{\partial \ell}{\partial \underline{y}}} \cdot \underset{1 \times n}{\underline{x}^T}$$

Intuitively by dimension matching

- In general, the simpler rule can be inferred via pattern intuition / looking at the dimensionality of the matrices, and these tensor derivatives need not be explicitly derived.
- Indeed, actually calculating these tensor derivatives, storing them, and then doing e.g., a tensor-vector multiply, is usually not a good idea for both memory and computation. In this example, storing all these zeros and performing the multiplications is unnecessary.
- If we know the end result is simply an outer product of two vectors, we need not even calculate an additional derivative in this step of backpropagation, or store an extra value (assuming the inputs were previously cached).



$\underline{y} = \underline{\underline{W}}\,\underline{x}$ $\quad \nabla_{\underline{x}}\,\underline{y} = \underline{\underline{W}}^T$



Regularizations and training neural networks

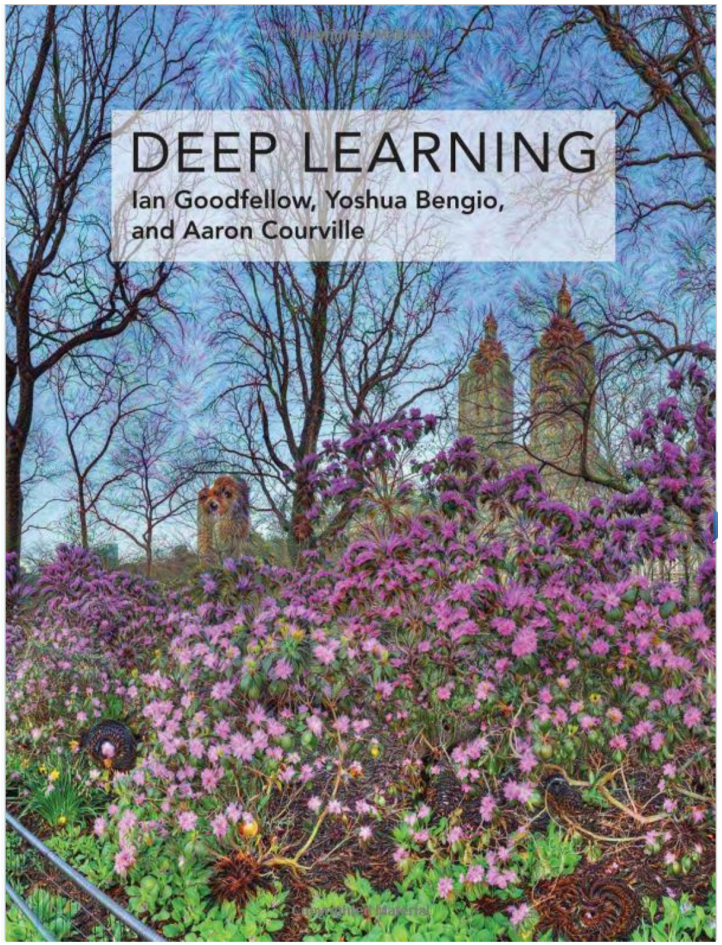
Next we will talk about specific techniques that aid in training neural networks. This lecture will focus on a few topics that are relevant for training neural networks well. The next lecture will then focus on optimization.

- Weight initialization in neural networks
- Batch normalization
- Regularizations
  - L1 + L2 normalization
  - Dataset augmentation
  - Model ensembles / Bagging
  - Dropout

Regularizations and training neural networks

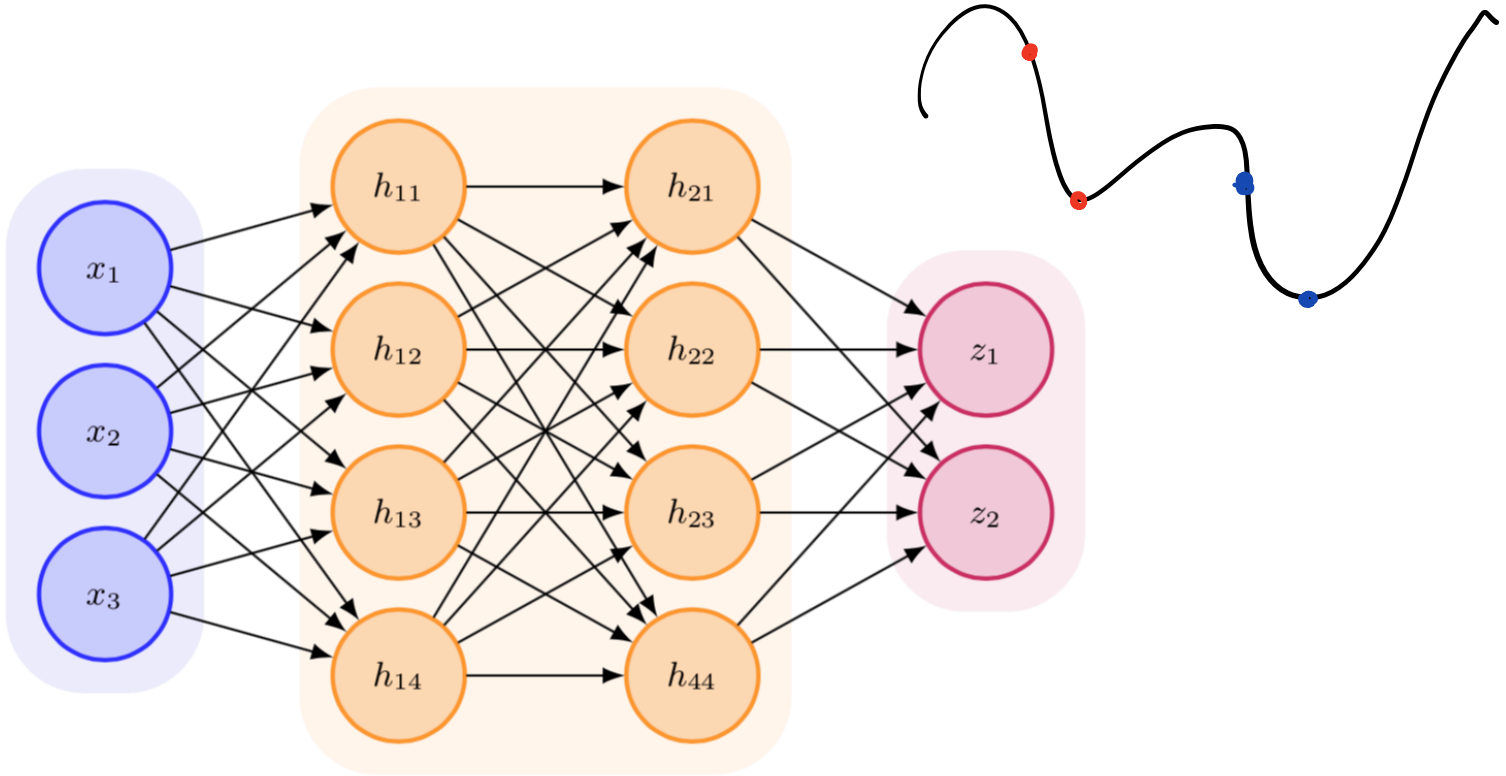
Reading:

Deep Learning, 7 (intro), 7.1, 7.2, 7.4, 7.5, 7.7, 7.9, 7.11, 7.12 (skim), 8.7.1



Initializations

→ Optimizing NN is nonconvex



Initializations matter.



Small random weight initialization

How about small random weight initializations?

The thought is that we don't set them large enough to bias training.

```
# Generate random data
X = np.random.randn(100,1000) # input is 100-dimensional, with 1000 data points
num_layers = 10
layer_sizes = [100] * num_layers # 10 hidden layers with 100 units each

# Forward propagation
Hs = [X]
Zs = [X]
Ws = []
for i in np.arange(num_layers):
    H = Hs[-1]

    # Initialize small weights
    W = np.random.randn(layer_sizes[i], H.shape[0]) * 0.01
    Z = np.dot(W, H)
    H = Z * (Z > 0)
    Hs.append(H)
    Ws.append(W)
    Zs.append(Z)
```

$W_j \sim \eta(0, \sigma^2)$

small

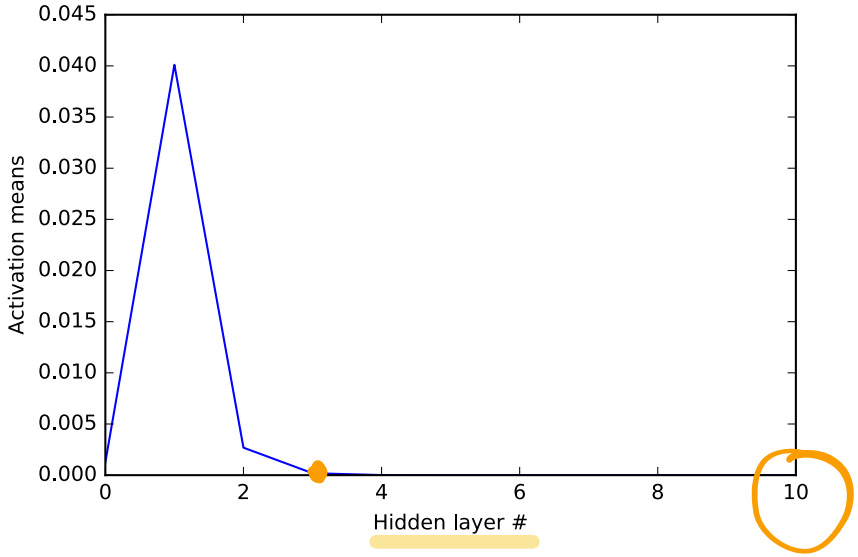
eg 0.01

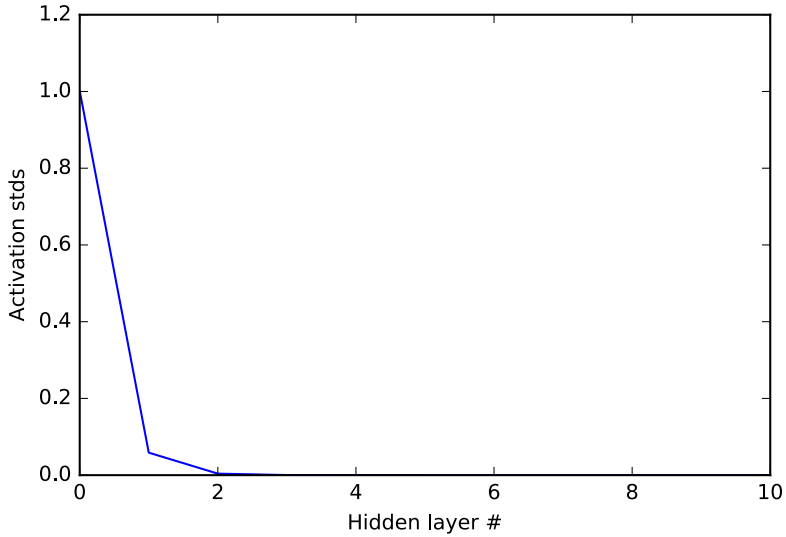
large

Small random weight initialization

Empirically, these initializations cause all activations to decay to zero.

Small initialization $\eta(0, 0.01)$

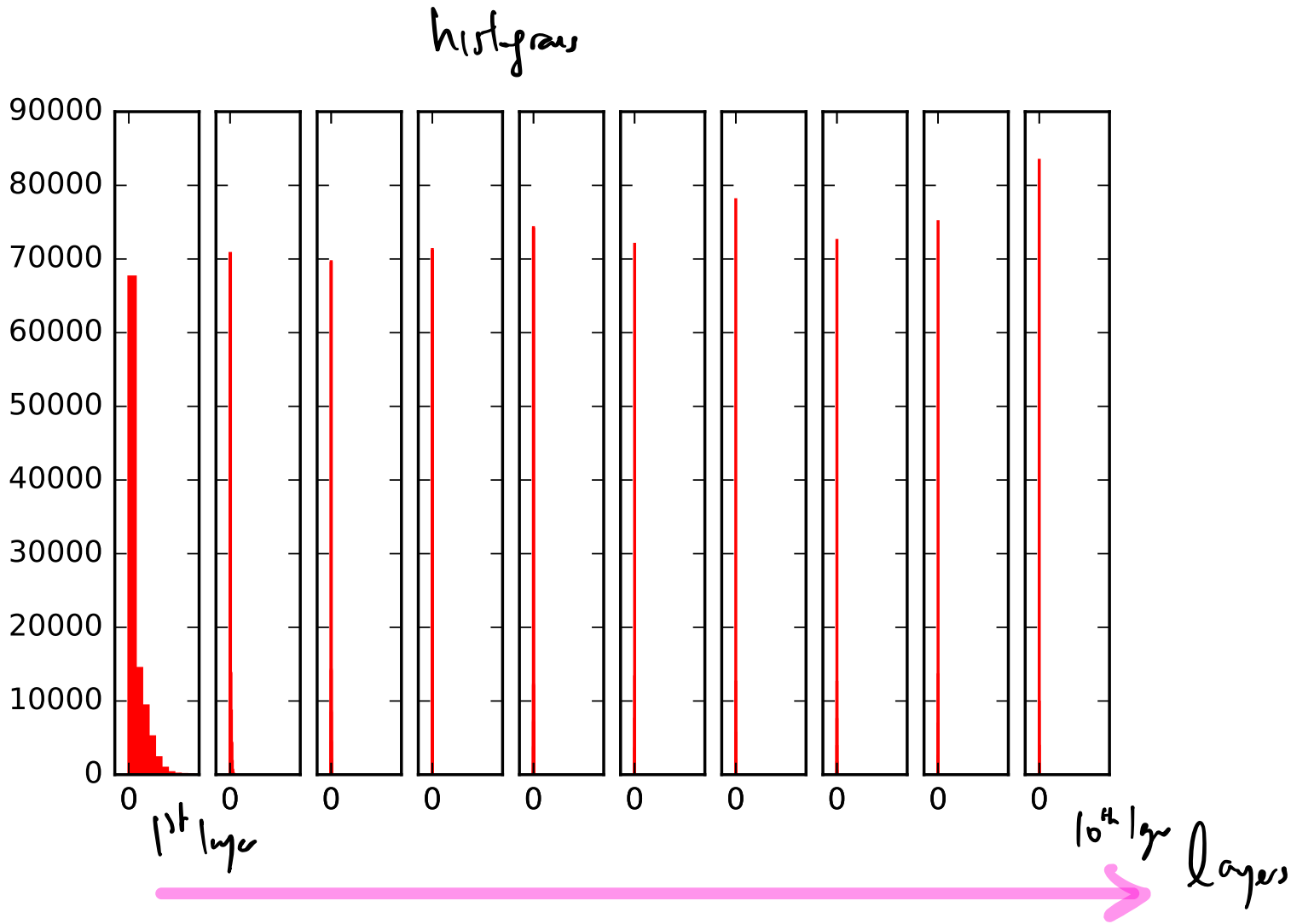




$\underline{h}^{(1)} = \underline{\underline{W}}^{(1)} \underline{x} + \underline{b}^{(1)} \quad \ldots \quad \underline{h}^{(10)} = \underline{\underline{W}}^{(10)} \underline{h}^{(9)} + \underline{b}^{(10)}$

Small random weight initialization

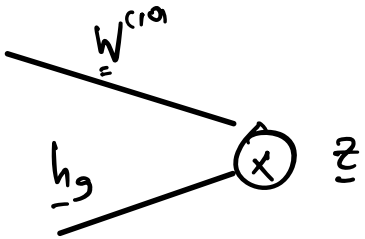
Distribution of each layer's activations:



Small random weight initialization

It is bad for learning if the output activations are close to zero. Why?

Think about the value of: $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$



$$\frac{\partial J}{\partial \underline{z}^{(l)}} \underline{h}_{l-1}^{T} \quad BP \approx 0$$



Small random weight initialization

It is bad for learning if the output activations are close to zero. Why?

Think about the value of: $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$

note that we sometimes use $J(\boldsymbol{\theta})$ or interchangeably use $\mathscr{L}(\boldsymbol{\theta})$

W vs θ

```
# Now backprop
dLdH = 100*np.random.randn(100, 1000) # 1000 losses for examples
loss_grads = [dLdH]
grads = []

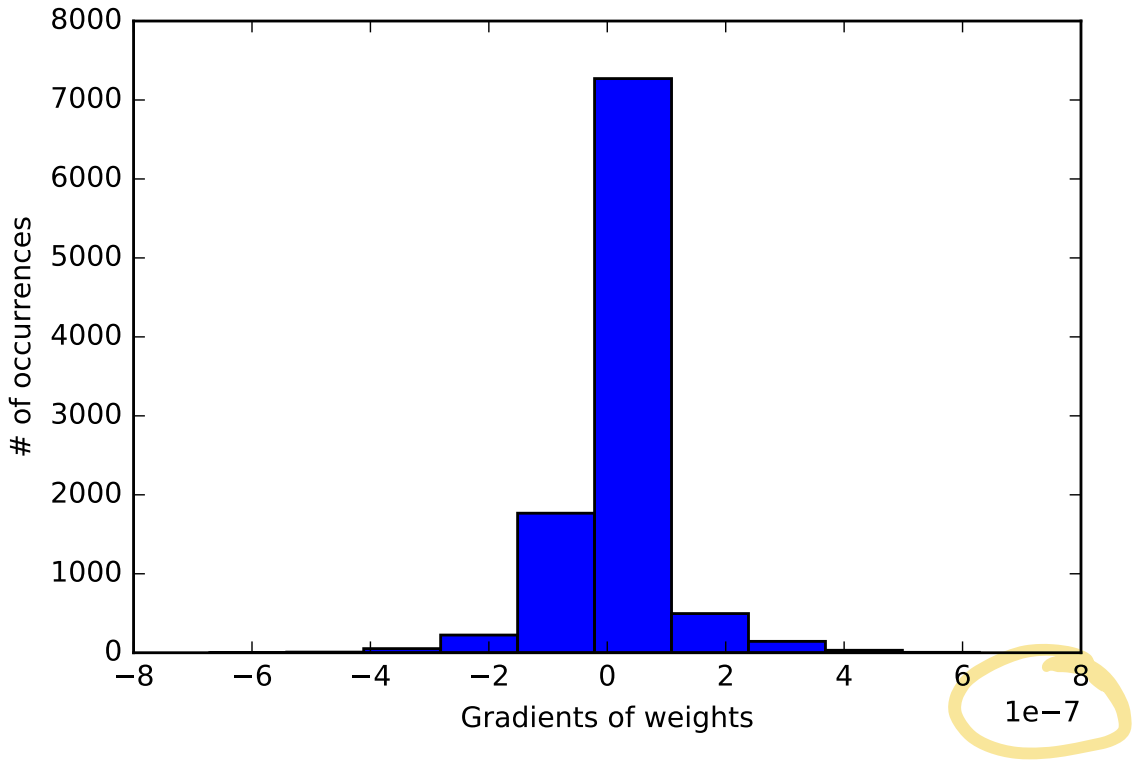
for i in np.flip(np.arange(1,11), axis=0):
    loss_grad = loss_grads[-1]
    dLdZ = loss_grad * (Z[i] > 0)
    grad_W = np.dot(dLdZ, Hs[i-1].T)
    grad_h = np.dot(Ws[i-1].T, dLdZ)
    loss_grads.append(grad_h)
    grads.append(grad_W)
grads = list(reversed(grads))
loss_grads = list(reversed(loss_grads))
```

Small random weight initialization

It is bad for learning if the output activations are close to zero. Why?

Think about the value of: $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ here we interchangeably use $\mathbf{W}$ and $\boldsymbol{\theta}$

Gradients for the last layer with respect to **W**:





Small random weight initialization

small # of layers

In practice, small weight initializations may be appropriate for smaller neural networks.

"shallow"



Large random weight initialization

How about larger random weight initializations?

```
# Generate random data
X = np.random.randn(100,1000) # input is 100-dimensional, with 1000 data points
num_layers = 10
layer_sizes = [100] * num_layers # 10 hidden layers with 100 units each

# Forward propagation
Hs = [X]
Zs = [X]
Ws = []
for i in np.arange(num_layers):
    H = Hs[-1]

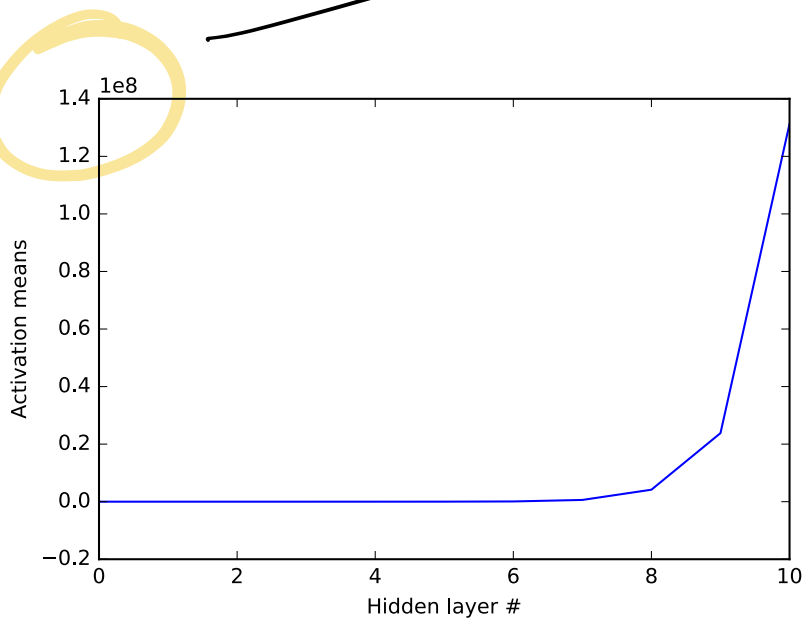
    # Initialize large weights
    W = np.random.randn(layer_sizes[i], H.shape[0]) * 1
    Z = np.dot(W, H)
    H = Z * (Z > 0)
    Hs.append(H)
    Ws.append(W)
    Zs.append(Z)
```

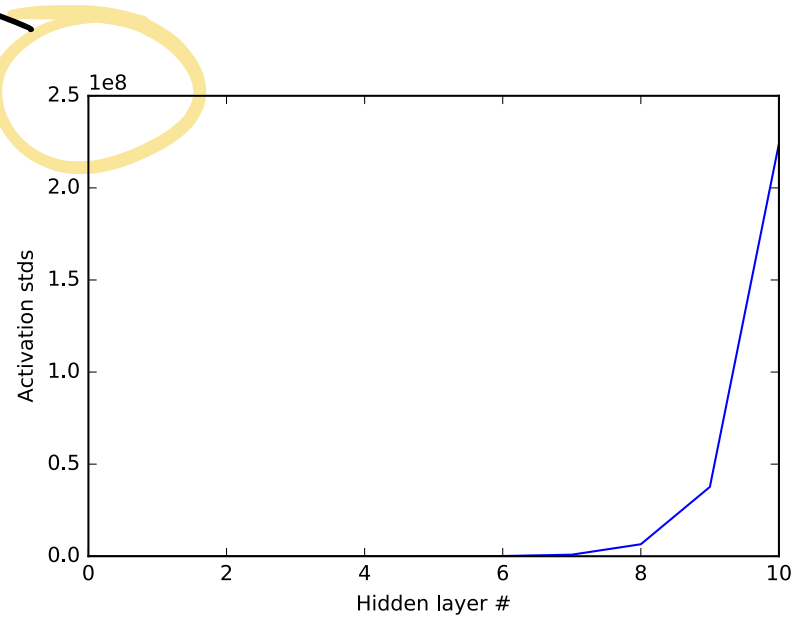
$\eta(0, 1)$

Large random weight initialization

Empirically, these cause the units to explode.

explode





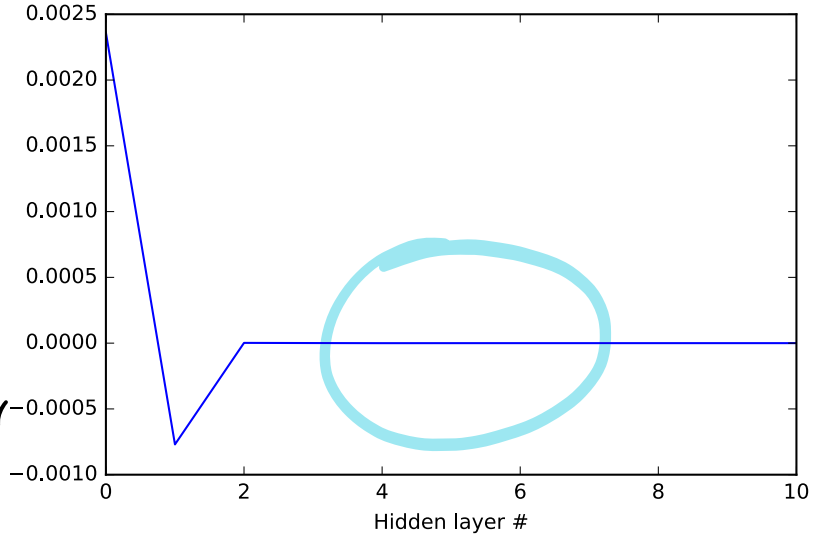
What happens to the gradients?

very noisy gradient

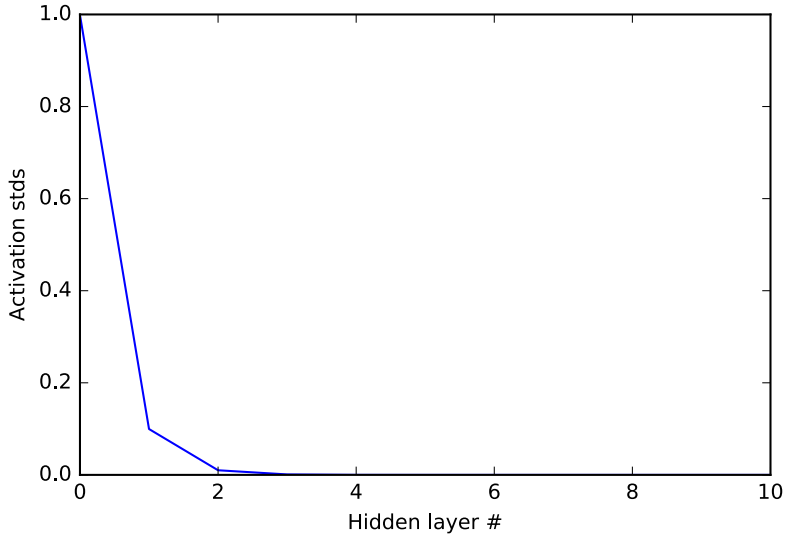
Again, this is a problem.

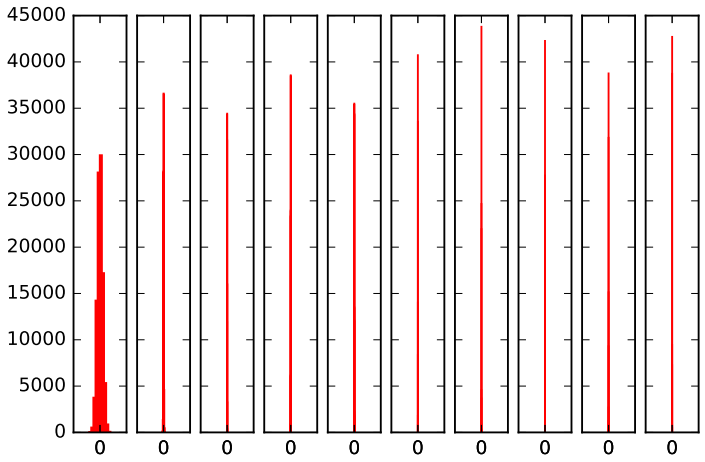
Will this problem occur for other activations?

What happens when we use the tanh() activation with small initialization?



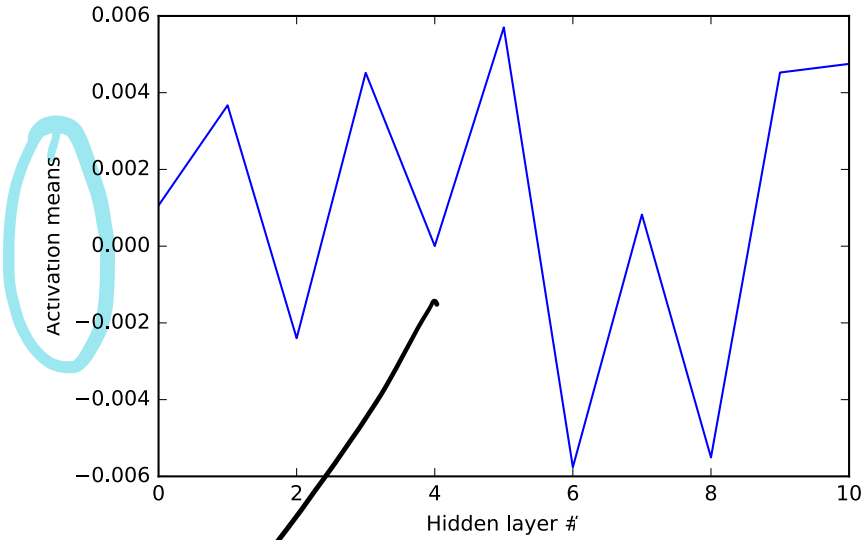
Small values of weight

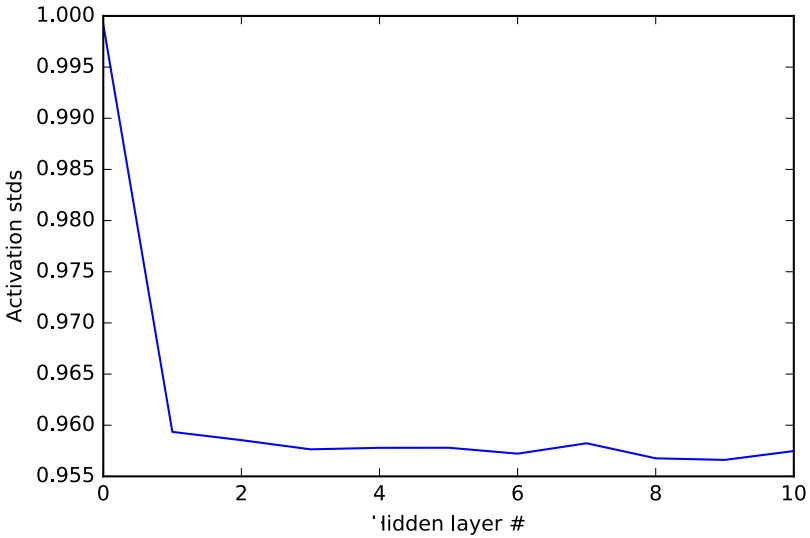




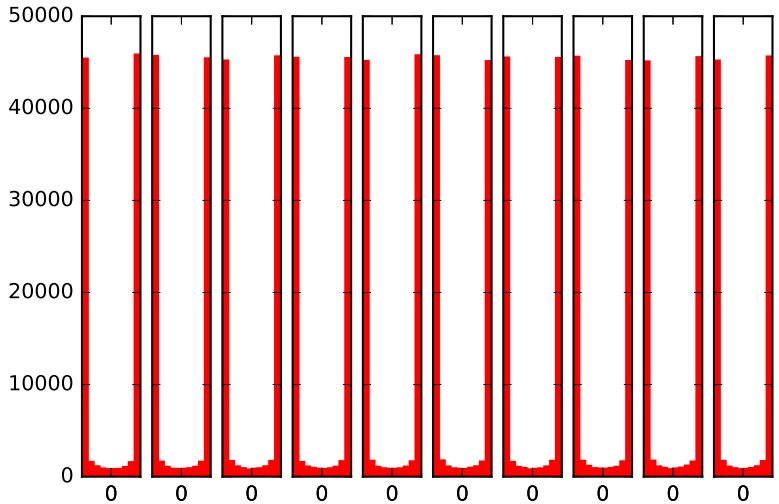
Will this problem occur for other activations?

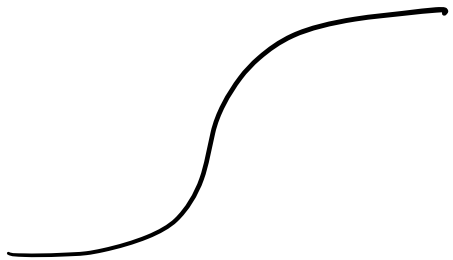
What happens when we use the tanh() activation with large initialization?





mean zero, variance 1







If not small and not large initialization, then what?

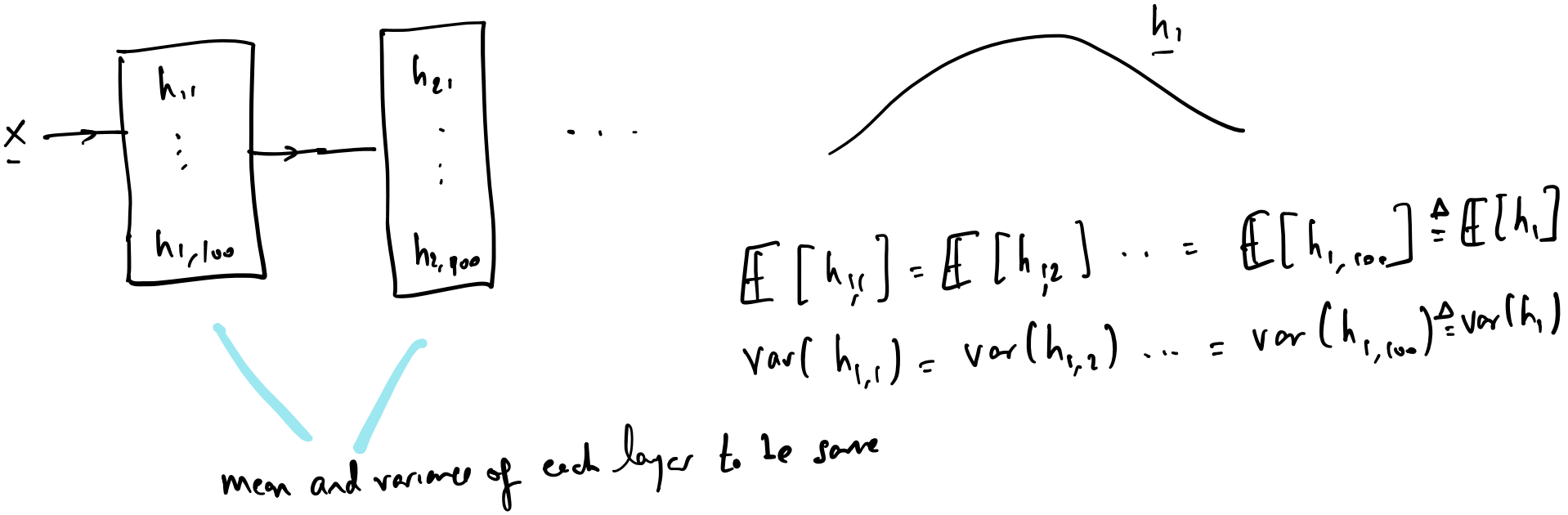
The next idea is to try an intermediate initialization.

But what intermediate value?

Xavier initialization

Xavier

One initialization is from Glorot and Bengio, 2011, referred to commonly as the Xavier initialization. It intuitively argues that the variance of the units across all layers ought be the same, and that the same holds true for the backpropagated gradients.





Xavier initialization

For simplicity, we assume the input has equal variance across all dimensions. Then, each unit in each layer ought to have the same statistics. For simplicity we'll denote h_i to denote a unit in the ith layer, but the variances ought be the same for all units in this layer.

Concretely, the heuristics mean that

$$\mathrm{var}(h_i) = \mathrm{var}(h_j)$$

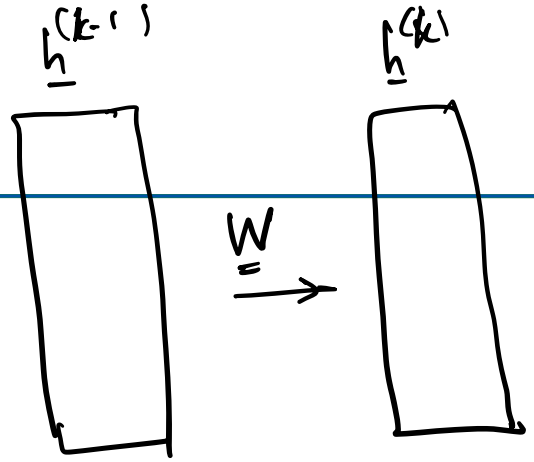
and

$$\mathrm{var}(\nabla_{h_i} J) = \mathrm{var}(\nabla_{h_j} J)$$



also want backpropagated variances to be same

will explain soon



Xavier initialization (cont.)

If the units are linear, then

GOAL: Calculate variance of $h^{(i)}$ and relate it to variance of $h^{(i-1)}$

$$h_i^{(k)} = \sum_{j=1}^{n_{\text{in}}} w_{ij} h_{i-1,j}^{(k-1)}$$

Further, if the w_{ij} and h_{i-1} are independent, and all the units in the $(i-1)$th layer have the same statistics, then using the fact that

$$\begin{aligned} \text{var}(wh) &= \mathbb{E}^2(w)\text{var}(h) + \mathbb{E}^2(h)\text{var}(w) + \text{var}(w)\text{var}(h) \\ &= \text{var}(w)\text{var}(h) \qquad \text{if } \mathbb{E}(w) = \mathbb{E}(h) = 0 \end{aligned}$$

then,

$$\text{var}(h_i) = \text{var}(h_{i-1}) \cdot \sum_{j=1}^{n_{\text{in}}} \text{var}(w_{ij})$$

$$\text{var}(w) \cdot n_{in} \approx 1$$

$$\Rightarrow \quad \text{var}(w) \approx \frac{1}{n_{in}}$$

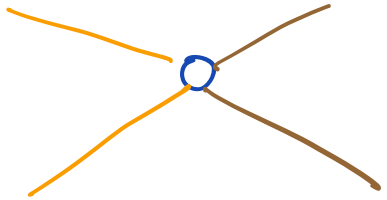
Xavier initialization

Now if the weights are identically distributed, then we get that for the unit activation variances to be equal,

$$\mathrm{var}(w_{ij}) = \frac{1}{n_{\mathrm{in}}}$$

for each connection j in layer i. The same argument can be made for the backpropagated gradients to argue that:

$$\mathrm{var}(w_{ij}) = \frac{1}{n_{\mathrm{out}}}$$





Xavier initialization

Xavier initialization (cont).

To incorporate both of these constraints, we can average the number of units together, so that

$$\text{var}(w_{ij}) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

Hence, we can initialize each weight in layer i to be drawn from:

$$\mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$



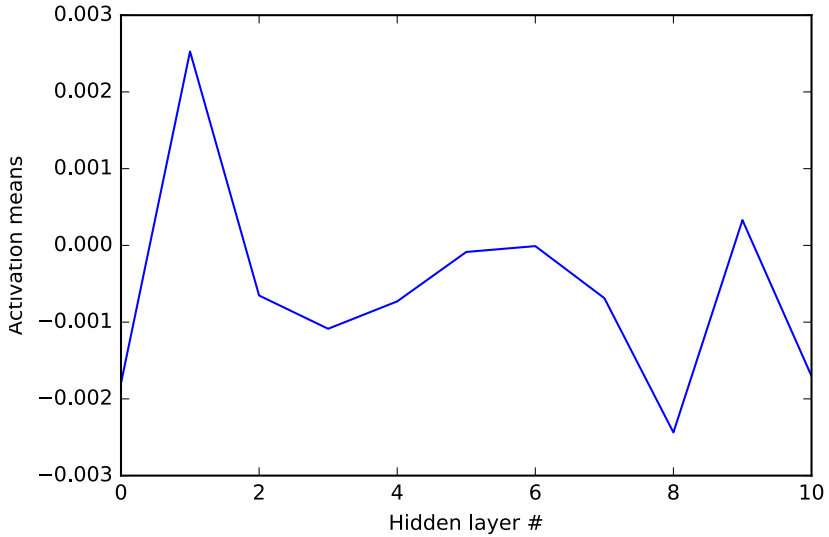
Xavier initialization with tanh

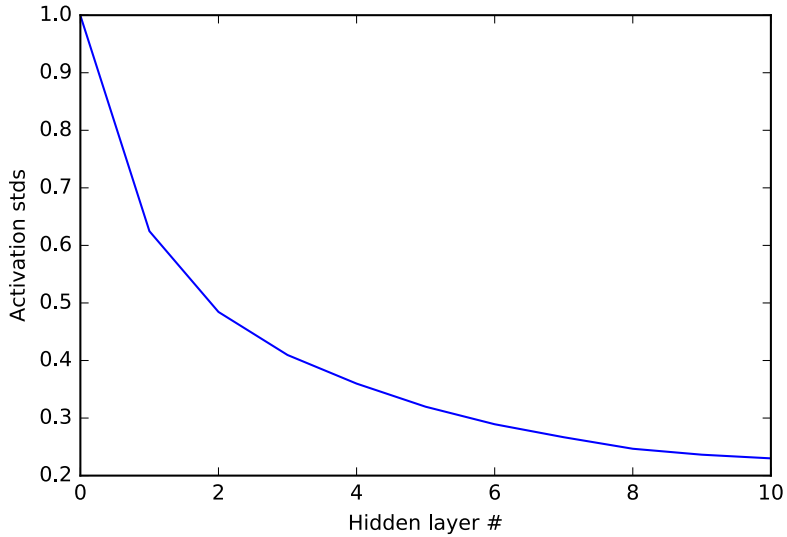
```
# Generate random data
X = np.random.randn(100,1000) # input is 100-dimensional, with 1000 data points
num_layers = 10
layer_sizes = [100] * num_layers # 10 hidden layers with 100 units each

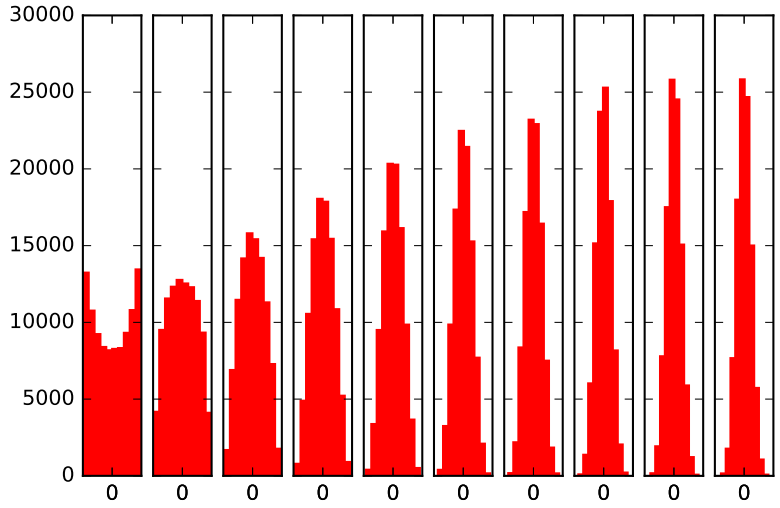
# Forward propagation
Hs = [X]
Zs = [X]
Ws = []
for i in np.arange(num_layers):
    H = Hs[-1]

    # Initialize Xavier
    W = np.random.randn(layer_sizes[i], H.shape[0]) * np.sqrt(2) / (np.sqrt(100 + 100))
    Z = np.dot(W, H)
    H = np.tanh(Z)
    Hs.append(H)
    Ws.append(W)
    Zs.append(Z)
```

Xavier initialization with tanh







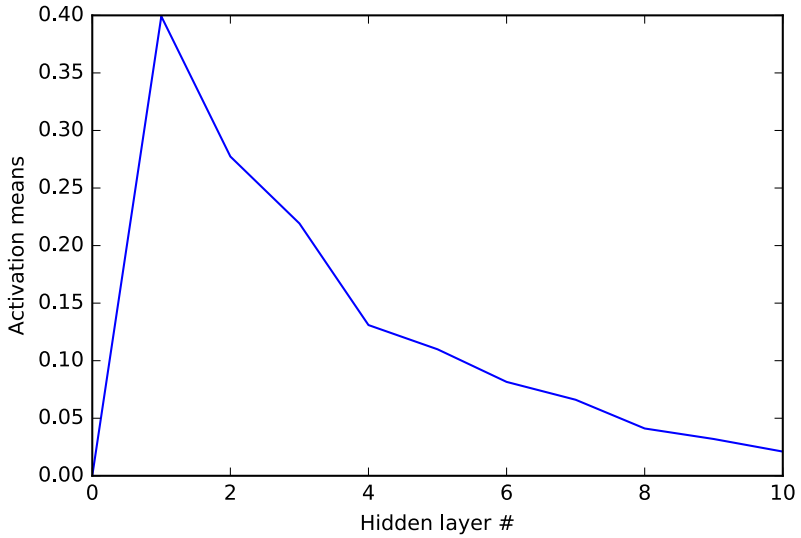
Xavier initialization

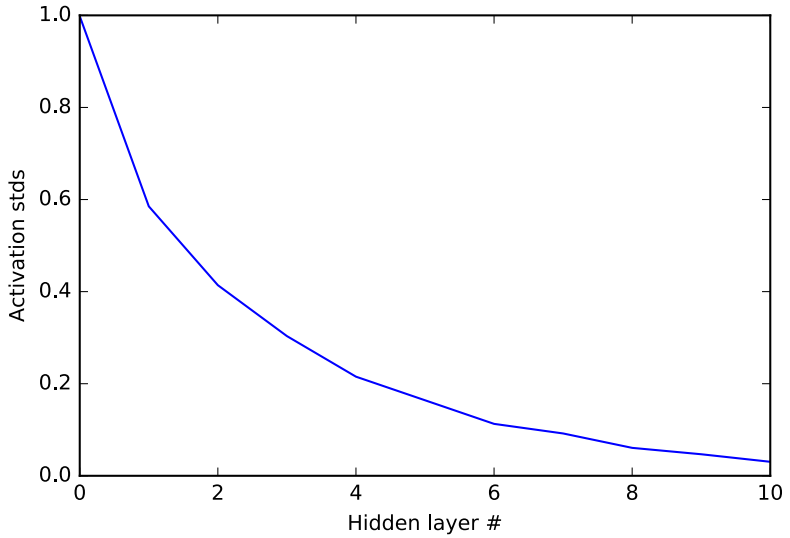
A few notes:

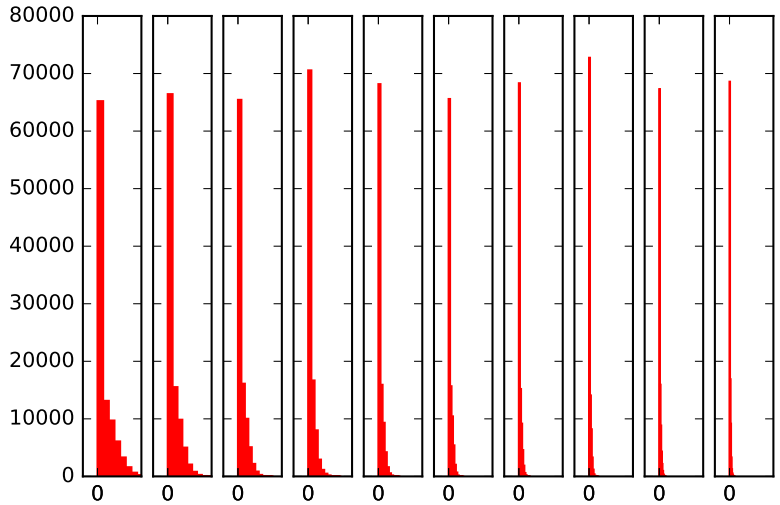
- The Xavier initialization (either one) typically leads to dying ReLU units, though it is fine with $\tanh$ units.
- He et al., 2015 suggest the normalizer $2/n_{\text{in}}$ when considering ReLU units. If linear activations prior to the ReLU are equally likely to be positive or negative, ReLU kills half of the units, and so the variance decreases by half. This motivates the additional factor of 2.
- Glorot and Bengio, 2015, ultimately suggest the weights be drawn from:

$$U\left(-\frac{\sqrt{6}}{n_{\text{in}}+n_{\text{out}}}, \frac{\sqrt{6}}{n_{\text{in}}+n_{\text{out}}}\right)$$

Xavier initialization with ReLU









He (MSRA) initialization with ReLU

MSR Asia

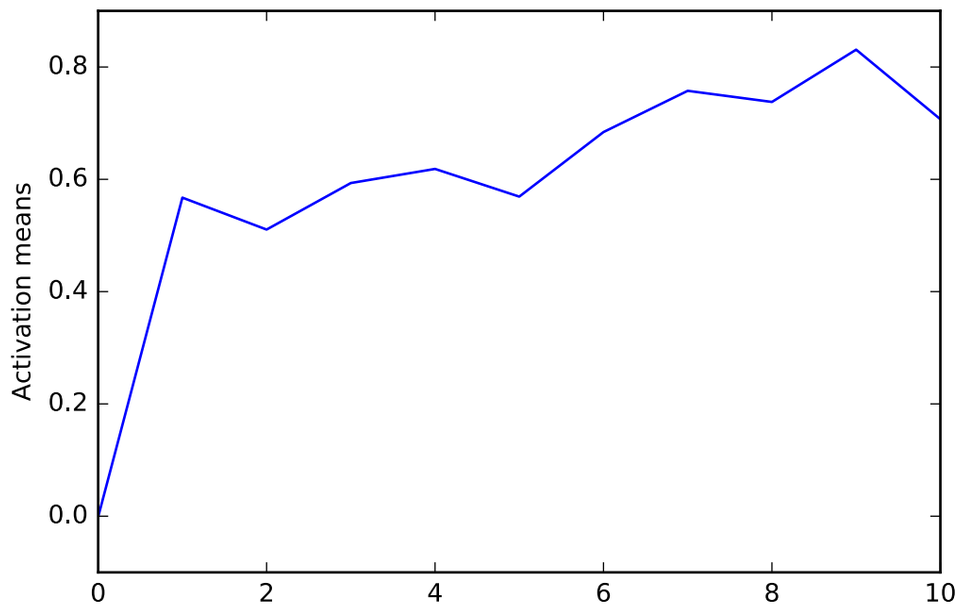
$2/n_{in}$ instead for ReLU

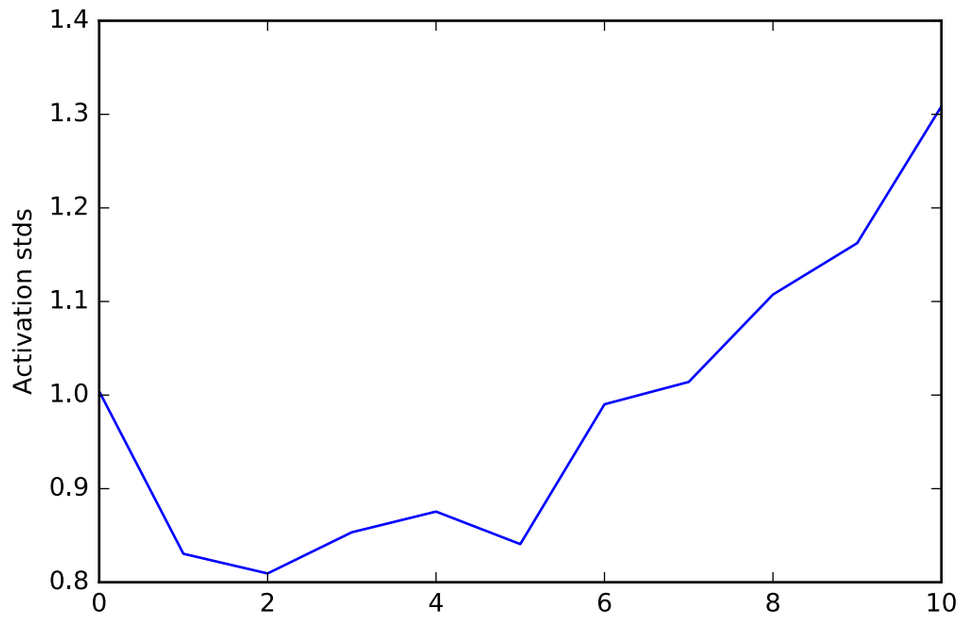
```
# Generate random data
X = np.random.randn(100,1000) # input is 100-dimensional, with 1000 data points
num_layers = 10
layer_sizes = [100] * num_layers # 10 hidden layers with 100 units each

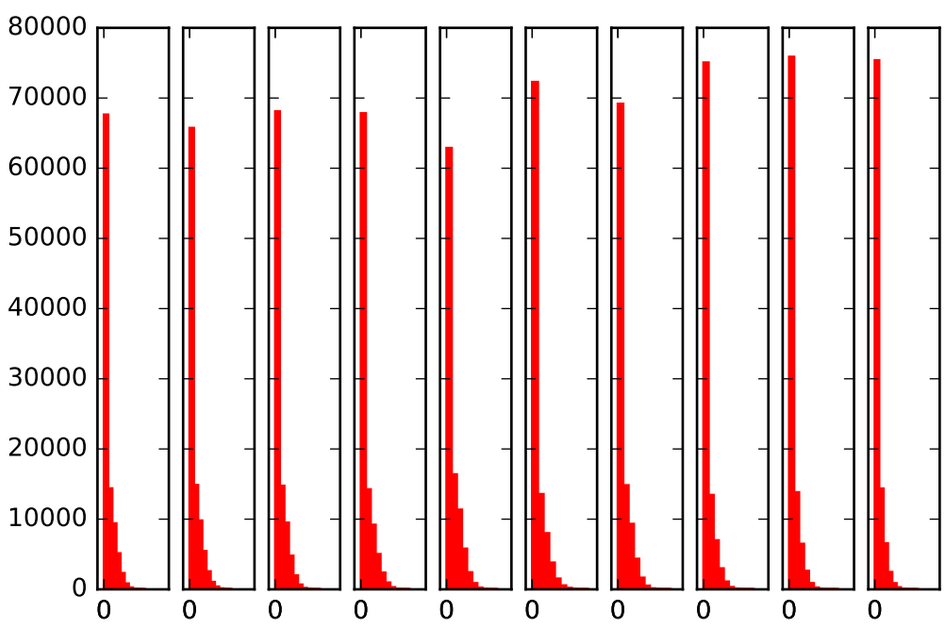
# Forward propagation
Hs = [X]
Zs = [X]
Ws = []
for i in np.arange(num_layers):
    H = Hs[-1]

    # Initialize Xavier
    W = np.random.randn(layer_sizes[i], H.shape[0]) * np.sqrt(2) / np.sqrt(100)
    Z = np.dot(W, H)
    H = Z * (Z>0)
    Hs.append(H)
    Ws.append(W)
    Zs.append(Z)
```

He (MSRA) initialization with ReLU



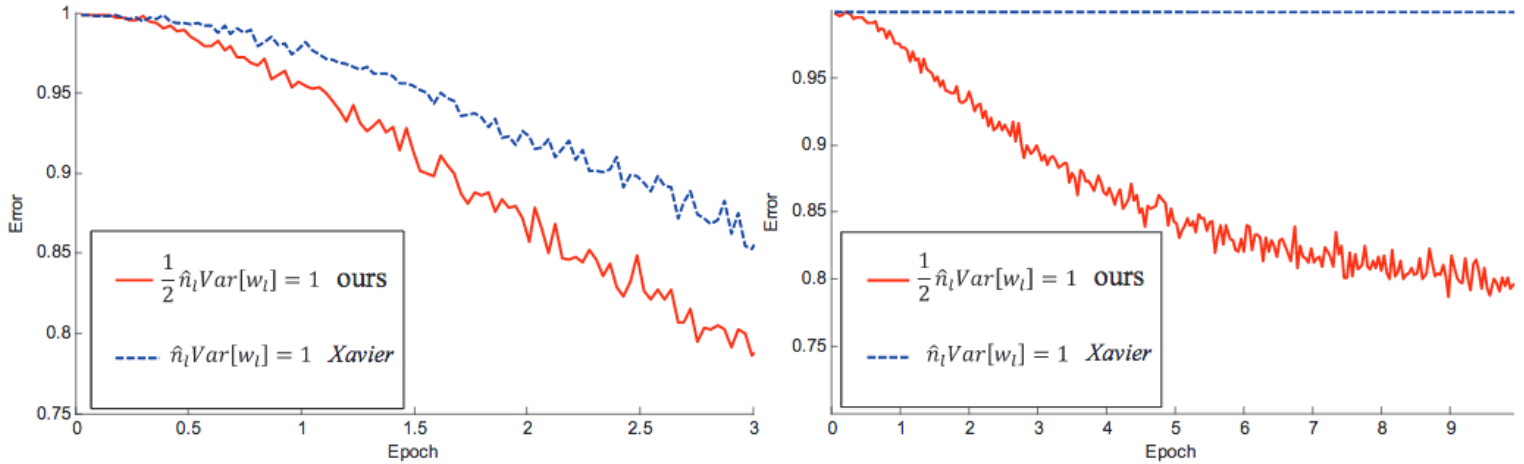




Initialization take home points

Initialization is a **very** important aspect of training neural networks and remains an active area of research.

A factor of two in the initialization can be the difference between the network learning well or not learning at all.



He et al., 2015

Sussillo and Abbott, "Random walk initialization for training very deep feedforward networks," 2014.
He et al., "Delving deep into rectifiers; surpassing human-level performance on ImageNet classification," 2015.
Mishkin and Matas, "All you need is a good init," 2015.



Initialization take home points

E.g., Mishkin and Matas, “All you need is a good init,” 2015.

Init method	maxout	ReLU	VLReLU	tanh	Sigmoid
LSUV	**93.94**	**92.11**	92.97	89.28	n/c
OrthoNorm	93.78	91.74	92.40	89.48	n/c
OrthoNorm-MSRA scaled	–	91.93	**93.09**	–	n/c
Xavier	91.75	90.63	92.27	**89.82**	n/c
MSRA	n/c†	90.91	92.43	89.54	n/c

Avoiding saturation, high variance activations, etc.

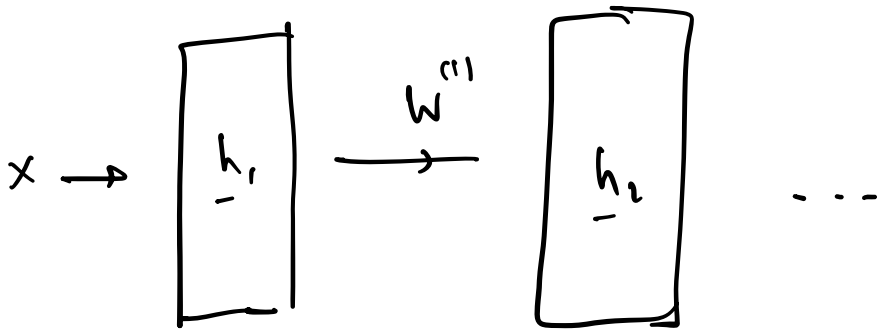
Ioffe - Szegedy '2015

We will motivate batchnorm through internal covariate shift, though new research suggests that batchnorm works primarily due to making the loss surface smoother.

An obstacle to standard training is that the distribution of the inputs to each layer changes as learning occurs in previous layers. As a result, the unit activations can be very variable. Another consideration is that when we do gradient descent, we're calculating how to update each paramer *assuming the other layers don't change*. But these layers may change drastically.

Internal covariance shift

↳ "distribution shift"



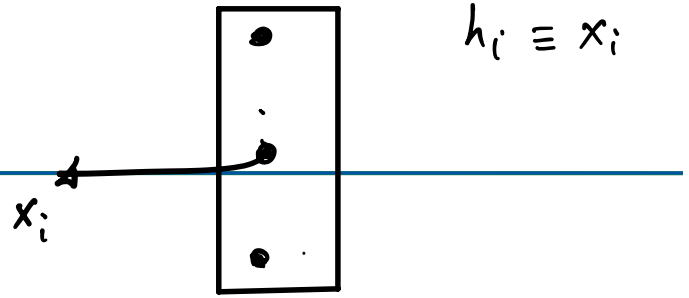
$\frac{\partial \ell}{\partial W_{SM}}$ → backpropagate

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \ell}{\partial W^{(l)}}$$



Batch normalization

The idea of batch-normalization is to make the output of each layer have unit statistics. Learning then becomes simpler because parameters in the lower layers do not change the statistics of the input to a given layer. This makes learning more straightforward.



Batch normalization (cont.)

(Ioffe and Szegedy, 2015), introduced batch normalization.

- Normalize the unit activations:

$$\hat{x}_i = \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$

~ 0 mean variance 1

hyperparameter not to get divide by 0

batch size

with

$$\mu_i = \frac{1}{m}\sum_{j=1}^{m} x_i^{(j)} \qquad \sigma_i^2 = \frac{1}{m}\sum_{j=1}^{m}\left(x_i^{(j)} - \mu_i\right)^2$$

and ε small.

- Scale and shift the normalized activations:

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

$\text{mean}(y_i) = \beta_i$

$\text{var}(y_i) = \gamma_i^2$

Importantly, the normalization and scale / shift operations are included in the computational graph of the neural network, so that they participate not only in forward propagation, but also in backpropagation.

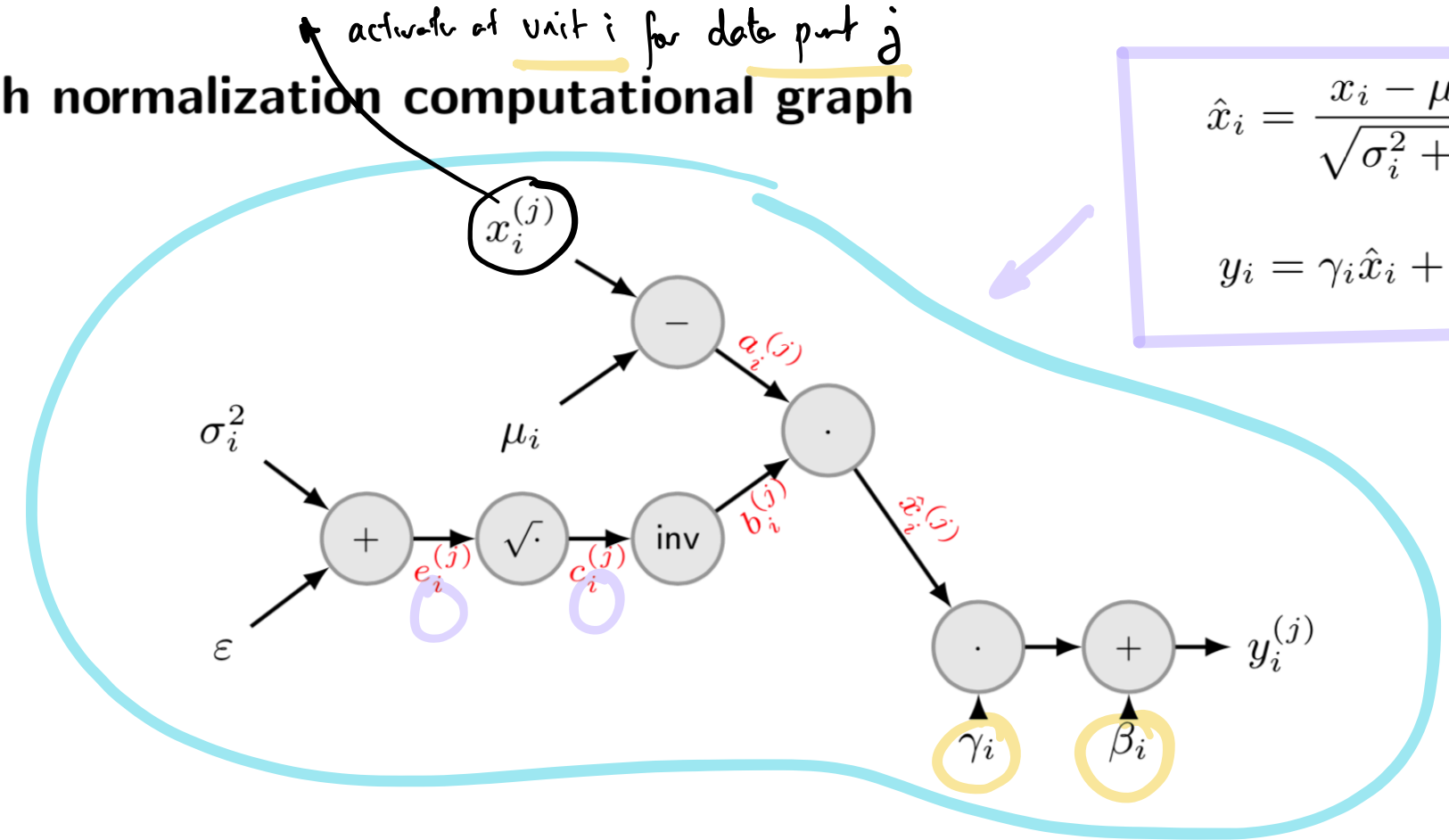
Batch normalization (cont.)

A few notes on batch normalization's implementation:

- The reason the scaling is on a per unit basis is primarily computational efficiency. It is possible to normalize the entire layer via $\Sigma^{-1/2}(\mathbf{x} - \mu)$ where μ, Σ are the mean and covariance of $\mathbf{x}$. However, this requires computation of a covariance matrix, its inverse, and the appropriate terms for backpropagation (including computation of the Jacobian of this transform).
- The scale and shift layer is inserted in case it is better that the activations not be zero mean and unit variance. As γ_i and β_i are parameters, it is possible for the network to rescale the activations. In fact, it could learn $\gamma_i = \sigma_i$ and $\beta_i = \mu_i$ to undo the normalization.

Batch normalization

Batch normalization computational graph



$$\hat{x}_i = \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$

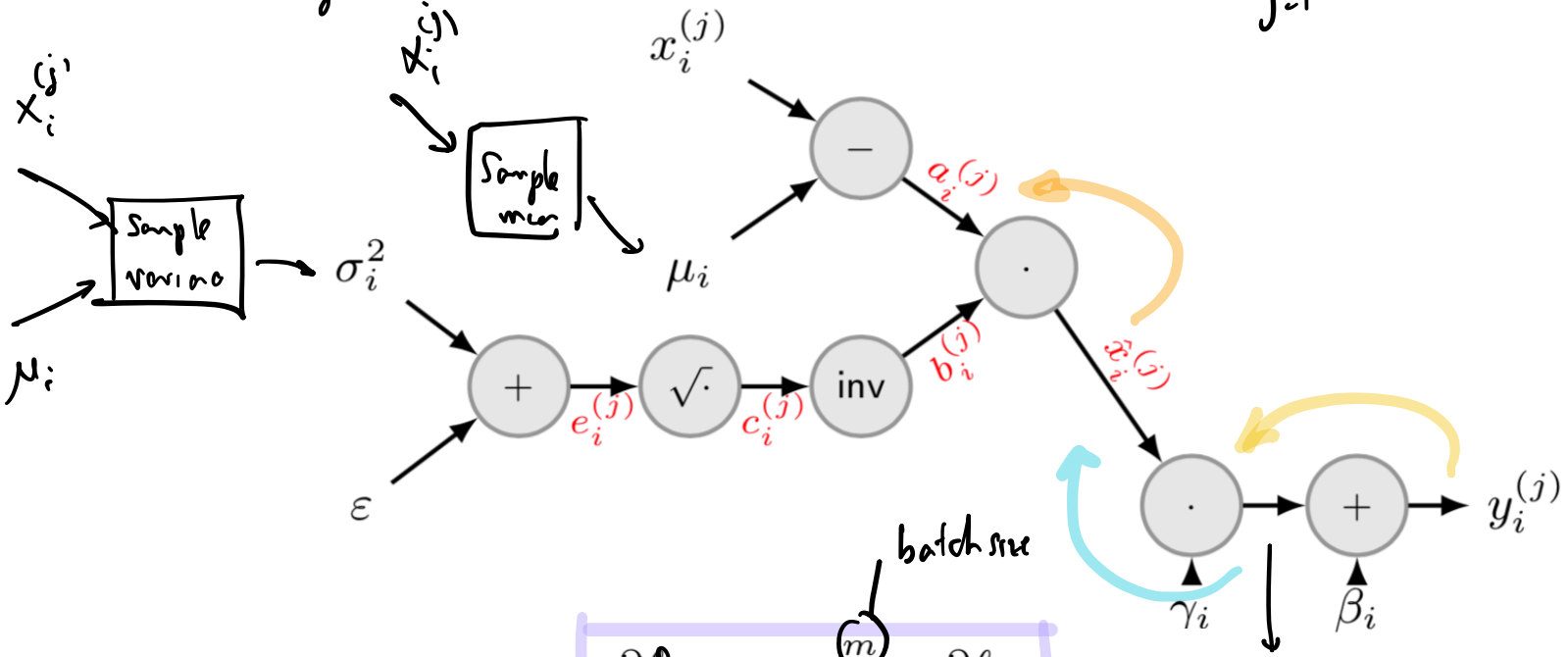
$$y_i = \gamma_i \hat{x}_i + \beta_i$$

Gradients from backprop on next page.

We will drop the subscript i for convenience.

Batch normalization

$$\sigma_i^2 = \frac{1}{m}\sum_{j=1}^{m}|x_i^{(j)} - \mu_i|^2 \qquad \mu_i = \frac{1}{m}\sum_{j=1}^{m} x_i^{(j)}$$



batch size

$$\frac{\partial \ell}{\partial \beta} = \sum_{j=1}^{m} \frac{\partial \ell}{\partial y^{(j)}}$$

$$\frac{\partial \ell}{\partial \gamma} = \sum_{j=1}^{m} \frac{\partial \ell}{\partial y^{(j)}} \hat{x}^{(j)}$$

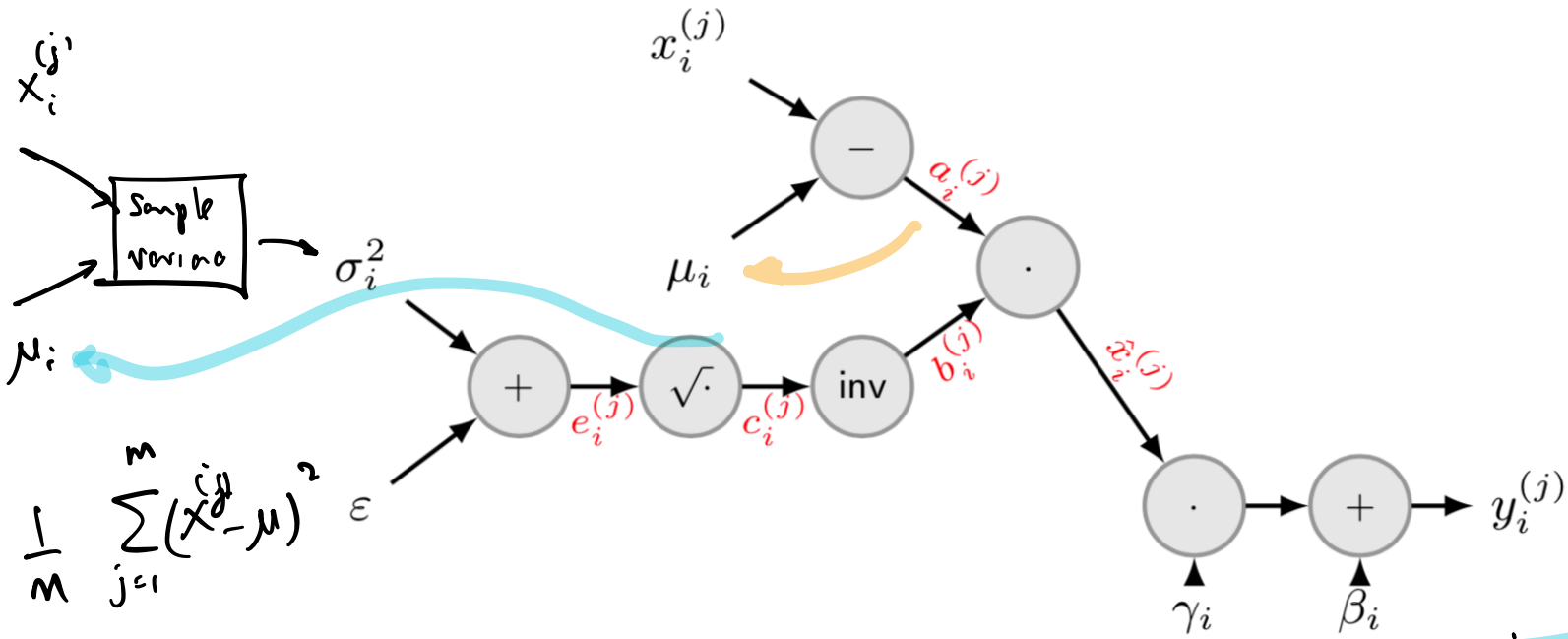
$$\frac{\partial \ell}{\partial \hat{x}^{(j)}} = \frac{\partial \ell}{\partial y^{(j)}} \gamma$$

$$\frac{\partial \ell}{\partial a^{(j)}} = \frac{1}{\sqrt{\sigma^2 + \epsilon}} \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

$\frac{\partial \ell}{\partial y^{(j)}}$ upstream known

$\frac{\partial \ell}{\partial y^{(j)}}$

Batch normalization



$$\sigma^2 = \frac{1}{m}\sum_{j=1}^{m}(x^{(j)}-\mu)^2$$

$$\frac{\partial \sigma^2}{\partial \mu} = \frac{1}{m}\sum_{j=1}^{m} 2(x^{(j)}-\mu)$$

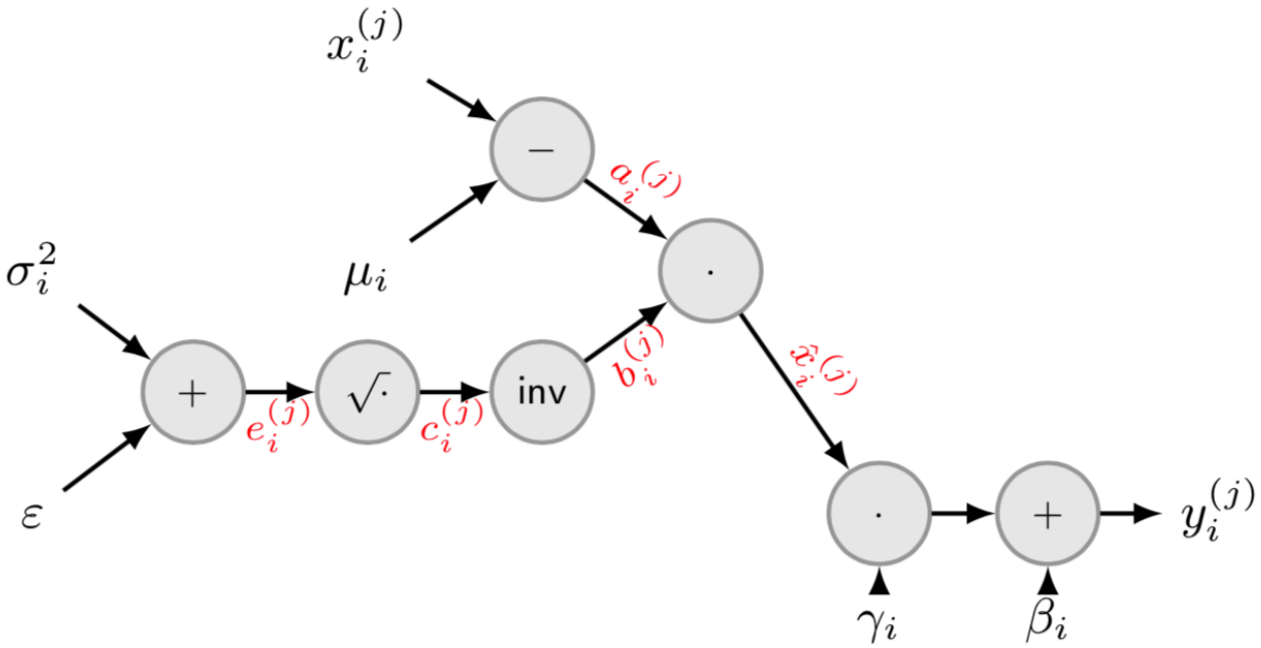
$$\frac{\partial \ell}{\partial \mu} = -\frac{1}{\sqrt{\sigma^2+\epsilon}}\sum_{j=1}^{m}\frac{\partial \ell}{\partial \hat{x}^{(j)}} - \frac{2}{m}\sum_{j=1}^{m}(x^{(j)}-\mu)\frac{\partial \ell}{\partial \sigma^2}$$

$$\frac{\partial \ell}{\partial b^{(j)}} = (x^{(j)}-\mu)\frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

$$\frac{\partial \ell}{\partial c^{(j)}} = -\frac{1}{\sigma^2+\varepsilon}(x^{(j)}-\mu)\frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

$$\frac{\partial \ell}{\partial e^{(j)}} = -\frac{1}{2}\frac{1}{(\sigma^2+\varepsilon)^{3/2}}(x^{(j)}-\mu)\frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

Batch normalization

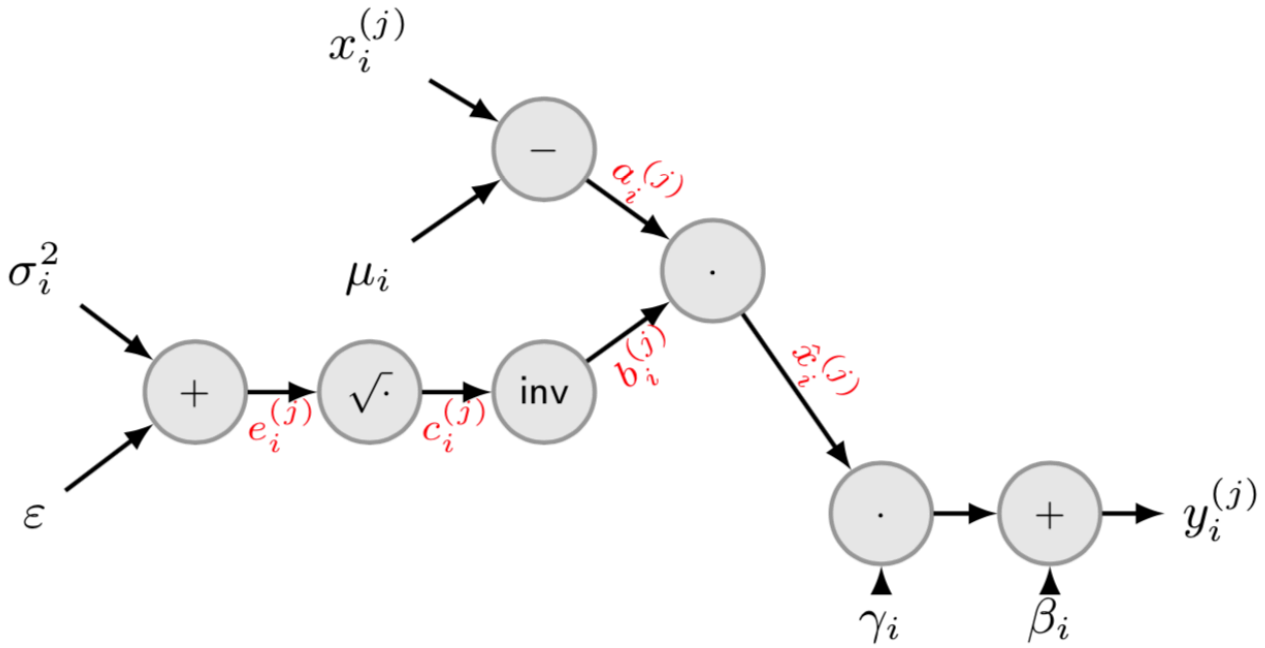


$$\frac{\partial \ell}{\partial a^{(j)}} = \frac{1}{\sqrt{\sigma^2+\epsilon}} \frac{\partial \ell}{\partial \hat{x}^{(j)}} \qquad \frac{\partial \ell}{\partial e^{(j)}} = -\frac{1}{2} \frac{1}{(\sigma^2+\varepsilon)^{3/2}} (x^{(j)} - \mu) \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

$$\frac{\partial \ell}{\partial \sigma^2} = \sum_{j=1}^{m} \frac{\partial \ell}{\partial e^{(j)}}$$

$$= \sum_{j=1}^{m} -\frac{1}{2} \frac{1}{(\sigma^2+\varepsilon)^{3/2}} (x^{(j)} - \mu) \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

Batch normalization



$$\frac{\partial \ell}{\partial a^{(j)}} = \frac{1}{\sqrt{\sigma^2+\epsilon}} \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

$$\frac{\partial \ell}{\partial e^{(j)}} = -\frac{1}{2} \frac{1}{(\sigma^2+\varepsilon)^{3/2}} (x^{(j)} - \mu) \frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

$$\frac{\partial \ell}{\partial x^{(j)}} = \frac{\partial \ell}{\partial a^{(j)}} + \frac{\partial \sigma^2}{\partial x^{(j)}} \frac{\partial \ell}{\partial \sigma^2} + \frac{\partial \mu}{\partial x^{(j)}} \frac{\partial \ell}{\partial \mu}$$

$$= \frac{1}{\sqrt{\sigma^2+\epsilon}} \frac{\partial \ell}{\partial \hat{x}^{(j)}} + \frac{2(x^{(j)} - \mu)}{m} \frac{\partial \ell}{\partial \sigma^2} + \frac{1}{m} \frac{\partial \ell}{\partial \mu}$$

Batch normalization layer

The batch normalization layer is typically placed right before the nonlinear activation. Hence, a layer of a neural network may look like:

$$\mathbf{h}_i = f\left(\text{batch-norm}\left(\mathbf{W}_i \mathbf{h}_{i-1} + \mathbf{b}_i\right)\right)$$

Empirically, batch normalization allows higher learning rates to be used.